TIKO — API Documentation

Oct 10, 2026 · @ki

1. Overview and conventions

The TIKO API is a small same-origin JSON API behind the web app: two sync routes carry almost all traffic because the app reads from its own device first, and the remaining routes handle identity, settings, notifications, export and deletion. It follows the Technical Specification (section 4) and Database Design; where they differ, fix the one that is wrong.

Topic	Rule
Base URL	`https://<app-domain>/api`, same origin as the web app; no cross-origin access (no CORS headers)
Format	JSON request and response bodies, UTF-8, `Content-Type: application/json`
Authentication	Session cookie set by sign-in (HttpOnly, Secure, SameSite=Lax); 30-day sliding expiry, 90-day absolute (proposed)
CSRF	Every write checks that `Origin` matches the app; Auth.js sign-in posts also need its CSRF token
Ids	UUID v7, generated by the client for synced entities
Timestamps	ISO 8601 UTC with `Z`, for example `2026-10-10T16:44:12.512Z`
Local dates	`local_date` as `YYYY-MM-DD` in the user's time zone at the moment of the action
Money	Integer paise in fields ending `_minor`; the currency comes from the user, default INR

Topic	Rule
Idempotency	Sync mutations carry an `op_id`; other POST routes accept an `Idempotency-Key` header, remembered 30 days
Versioning	No version in the path. Sync requests carry `schema_v`; an older client than the minimum gets `426 client_too_old` and reloads
Guests	Guests never call the API; their data stays on the device until first sign-in, then goes through `/api/sync/merge-guest`

Route index

Route	Method	Auth	Purpose
/api/auth/signin/email	POST	None	Send a magic link
/api/auth/callback/email	GET	None	Verify the magic link
/api/auth/signin/google	POST	None	Start Google sign-in
/api/auth/session	GET	Cookie	Current session
/api/auth/signout	POST	Cookie	End the session
/api/sync/push	POST	Cookie	Apply a batch of mutations
/api/sync/pull	GET	Cookie	Read changes since a cursor
/api/sync/merge-guest	POST	Cookie	Merge a guest's device data
/api/me	GET, PATCH	Cookie	Profile, limits, settings
/api/me/notifications	GET, PATCH	Cookie	Reminder time, tone, quiet hours, channels
/api/push/subscribe	POST, DELETE	Cookie	Add or remove a web push subscription
/api/export	GET	Cookie	Download all data
/api/account	DELETE	Cookie	Start account deletion
/api/cron/*	POST	Secret	Scheduled jobs

Route	Method	Auth	Purpose
/api/billing/*	POST, GET	Cookie or signature	Plus plan (V1)

The `/api/me/notifications` route is an addition: the Technical Specification does not list a route for reminder settings (FR-NOT-01, FR-SET-04), and they are not synced rows, so they need one.

2. Authentication

Sign-in has no passwords: an email magic link or Google. The routes are the standard Auth.js ones (or the Supabase Auth equivalents if Supabase is chosen), so only behavior TIKO depends on is specified here (FR-ACC-02 to 04).

Send a magic link

```
POST /api/auth/signin/email
Content-Type: application/json

{ "email": "aarav@example.com", "csrfToken": "<from GET /api/auth/csrf>" }
```

```
200 OK
{ "ok": true }
```

- The response is always 200, whether or not the address has an account, so the route cannot be used to find registered emails.
- The link is sent through Resend and should arrive in under 60 seconds (story A3).
- The token is a random 32-byte value stored hashed; it expires after 15 minutes and works once (FR-ACC-04).
- Limits: 5 requests per 10 minutes per email, 20 per hour per IP; beyond that, `429 rate_limited` with `Retry-After`.

Open the magic link

`GET /api/auth/callback/email?token=...&email=...` verifies the token, creates the user if new, sets the session cookie and redirects to the app. An expired, used or unknown token redirects to `/signin?error=link_invalid`; the app then offers to send a new link.

Google sign-in

`POST /api/auth/signin/google` starts the OAuth flow and `GET /api/auth/callback/google` completes it, then sets the same session cookie. An existing account with the same verified email is linked, not duplicated.

Current session

```
GET /api/auth/session
```

```
{ "user": { "id": "0191f3c2-6a41-7c3e-9b1d-2f4e8a7c5d10", "email":
"aarav@example.com", "name": "Aarav" },
  "expires": "2026-11-09T16:44:12.000Z" }
```

No session returns `{}` with status 200, so the client can call this on every start.

Sign out. `POST /api/auth/signout` revokes the session on the server and clears the cookie. The device keeps its local data.

After first sign-in. If the person used guest mode, the client calls `/api/sync/merge-guest` once before its first normal push (see section 3).

3. Sync API

The device is the source of truth; these routes copy its changes to the server and bring back changes from other devices (FR-SYN-02 to 06). Reading screens never calls the API.

POST /api/sync/push

Applies a batch of mutations, in order. Each mutation is independent: one rejection does not stop the others.

```
POST /api/sync/push

{
  "device_id": "0191f3c2-0000-7000-8000-00000000d001",
  "schema_v": 1,
  "ops": [
    {
      "op_id": "0191f3c2-6a41-7c3e-9b1d-2f4e8a7c5d11",
      "entity": "habit_log",
      "id": "0191f3c2-6a41-7c3e-9b1d-2f4e8a7c5d12",
      "action": "upsert",
      "updated_at": "2026-10-10T16:44:12.512Z",
```

```
      "data": { "habit_id": "0191f3c2-5a00-7000-8000-0000000000a1", "value":
1,
                "local_date": "2026-10-10", "logged_at": "2026-10-
10T16:44:12.512Z" }
    },
    {
      "op_id": "0191f3c2-6a41-7c3e-9b1d-2f4e8a7c5d13",
      "entity": "habit_log",
      "id": "0191f3c2-6a41-7c3e-9b1d-2f4e8a7c5d09",
      "action": "delete",
      "updated_at": "2026-10-10T16:44:20.100Z"
    }
  ]
}
```

```
200 OK
{
  "results": [
    { "op_id": "0191f3c2-6a41-7c3e-9b1d-2f4e8a7c5d11", "status": "applied" },
    { "op_id": "0191f3c2-6a41-7c3e-9b1d-2f4e8a7c5d13", "status": "applied" }
  ],
  "changes": [ /* rows the server changed or recomputed, same shape as pull
*/ ],
  "cursor": 48213,
  "server_time": "2026-10-10T16:44:13.040Z"
}
```

Field	Rule
`ops`	1 to 200 mutations, body up to 1 MB; larger batches get `413 payload_too_large` and are split by the client
`entity`	One of the names in section 4
`action`	`upsert` (create or update) or `delete` (tombstone); journal and xp entities cannot be hard-deleted
`updated_at`	Client time of the edit; the server clamps anything more than 5 minutes in the future

Field	Rule
Order	Send in creation order so parents (a habit) come before children (its logs)

Result statuses

Status	Meaning	Client action
`applied`	Stored	Remove from outbox
`duplicate`	`op_id` already applied earlier	Remove from outbox
`stale`	The server holds a newer edit of the row (latest wins); the row is returned in `changes`	Adopt the server row, remove from outbox
`rejected`	Invalid or not allowed; carries `error.code`	Do not retry unchanged; show a calm message if it concerns the user
`retry`	Temporary problem, such as a parent row that has not arrived yet	Keep in outbox, retry later

Rejection codes inside a result: `validation_failed`, `goal_limit_reached` (more than 5 active goals, or above the plan limit, with a message the app shows to explain the limit, FR-GOL-03), `habit_limit_reached`, `log_too_old` (edit or delete beyond 7 days, FR-HAB-10), `missing_parent`. Rejected mutations never delete anything on the device (FR-SYN-06).

After applying, the server reruns the shared engine for affected goals and returns recomputed goal rows in `changes`. If the device already showed a celebration, the client adopts the server state quietly (FR-SYN-05).

Journal entries. The client sends the plain `body` over HTTPS; the server encrypts it before storing and returns it decrypted to the owner. The ciphertext columns are never exposed by the API.

GET /api/sync/pull

Returns rows changed since a cursor, across all synced entities.

```
GET /api/sync/pull?cursor=48213&limit=500&schema_v=1
```

```
200 OK
{
  "changes": [
    { "entity": "goal", "id": "0191f3c2-7000-7000-8000-00000000a001", "seq":
48220,
      "deleted": false,
      "data": { "type": "daily_target", "status": "completed", "progress":
56, "target": 56 } }
  ],
  "cursor": 48220,
  "has_more": false
}
```

- `cursor` starts at `0` on a new device; `limit` is 1 to 500, default 500. If `has_more` is true, call again with the returned `cursor`.
- Re-pull from `cursor - 100` on each run and ignore rows already held (same id and `seq`); this covers sequence values that commit out of order.
- `410 cursor_too_old` means the device was unseen for over 90 days and tombstones are gone; the client does a full resync from `cursor=0`.
- Deleted rows come back with `"deleted": true` and no `data`.

POST /api/sync/merge-guest

Called once, after a person who used guest mode signs in. Same body as push, with the whole device dataset, and the same per-mutation results.

```
POST /api/sync/merge-guest

{ "device_id": "...", "schema_v": 1, "ops": [ /* every row on the device,
parents first */ ] }
```

```
200 OK
{
  "results": [ /* as in push */ ],
  "id_map": { "habit": { "0191f3c2-5a00-7000-8000-0000000000a1": "0191a000-
1111-7000-8000-0000000000b7" } },
  "summary": { "applied": 142, "duplicate": 0, "stale": 3, "rejected": 0 },
  "cursor": 48230
}
```

- Ids are client-generated, so retrying the call never duplicates rows (FR-ACC-03).
- Core habits match on `core_key`. If the account already has "water", the guest's water habit is merged into it: `id_map` lists the replacement id, the server rewrites the guest's logs and goals to point at it, and the client swaps the ids locally.
- When both sides hold the same row, the later `updated_at` wins; rows that exist on only one side are kept (union, nothing is overwritten wholesale).
- Plan and goal limits are not applied during a merge, so a guest never loses data on sign-in; extra goals beyond the cap are archived, not deleted.

4. Entity payloads

These are the `entity` names and `data` shapes used by push, pull and merge. They mirror the Database Design; `user_id`, `created_at`, `deleted_at` and `seq` are set by the server and are not sent in `data`. Unknown fields are rejected. A field marked `?` is optional.

Entity	Fields in `data`	Notes
`habit`	`core_key?`, `name`, `icon`, `mode` (tap, check, timer, number), `unit?`, `target` (> 0), `kind` (build, limit), `cadence` (daily, weekly_total), `paused_until?`, `archived_at?`, `sort_order`	Archived habits keep their history; one row per `core_key` per user
`habit_log`	`habit_id`, `value` (>= 0), `local_date`, `logged_at`, `source?`	Edit or delete only within 7 days
`goal`	`type` (daily_target, streak, weekly_total, count, limit, savings), `title`, `habit_id?`, `category_id?`, `target`, `period_days?`, `start_date`, `end_date?`, `status`, `treat_id?`, `softened_factor?`, `claimed_at?`	`progress`, `completed_at`, `nudged_50` and `nudged_90` are recomputed by the server; there is no failed status
`treat`	`name`, `starter_key?`, `photo_ref?`, `cost_minor?`, `kind` (free, paid)	Photo upload is separate (see below)

Entity	Fields in `data`	Notes
`goal_contribution`	`goal_id`, `amount_minor` (> 0), `local_date`	Savings goals, V1
`category`	`name`, `default_key?`, `monthly_limit_minor?`	Eight defaults created on first sync
`expense`	`amount_minor` (> 0), `category_id`, `local_date`, `note?`, `is_income`, `is_treat`, `treat_id?`, `recurring_id?`	Income and paid treats are excluded from safe-to-spend
`budget`	`month` (first day, `YYYY-MM-01`), `amount_minor`, `treat_budget_minor?`	One per month
`recurring_expense`	`amount_minor`, `category_id`, `day_of_month` (1 to 31), `active`	P1; the server posts the expenses
`mood_entry`	`score` (1 to 5), `energy` (low, medium, high), `tags?` (max 3), `note?` (one line, 140 characters or fewer), `local_date`, `slot?`	One per date and slot
`journal_entry`	`kind` (free, gratitude), `body`, `local_date`	Plain `body` in transit; stored encrypted; never in analytics
`xp_event`	`source_type`, `source_id`, `points` (> 0)	Upsert only; unique per source, so retries cannot award twice

Treat photos. Photos are not sent inside sync. The client resizes to at most 1280 px WebP and uploads it with `POST /api/photos` (an upload route to be added to the Technical Specification; it returns a `photo_ref` key), then sets `photo_ref` on the treat. Offline, the photo waits on the device and the upload is queued. Starter treats use bundled images and need no upload.

Pulled-only values. Goal `progress`, `completed_at` and the nudge flags come back in `changes` after the server recomputes. The client may compute them locally first for instant feedback, but the server value wins.

5. Account, settings and data

GET and PATCH /api/me

```
GET /api/me
```

```
{
  "id": "0191f3c2-6a41-7c3e-9b1d-2f4e8a7c5d10",
  "email": "aarav@example.com",
  "name": "Aarav",
  "time_zone": "Asia/Kolkata",
  "currency": "INR",
  "plan": "free",
  "limits": { "max_habits": 5, "max_goals": 2, "max_active_goals": 5 },
  "softened_until": null,
  "softening_on": true,
  "streaks_hidden": false,
  "delete_requested_at": null
}
```

`PATCH /api/me` accepts any of `name`, `time_zone` (valid IANA name), `currency` (ISO 4217 code), `streaks_hidden`, `softening_on` and returns the updated object (FR-ACC-08, FR-ENG-09, FR-SET-04). Email and plan cannot be changed here. Changing the time zone affects only future days; stored `local_date` values are never rewritten.

GET and PATCH /api/me/notifications

```
{ "reminder_time": "20:30", "tone": "gentle", "quiet_start": "22:00",
"quiet_end": "08:00",
  "push_on": true, "email_on": true }
```

- `tone` is `gentle`, `playful` or `minimal`; times are `HH:MM` in the user's time zone (FR-NOT-01, 02).
- Setting `reminder_time` for the first time is the moment the app asks the browser for push permission, never earlier (FR-ACC-06).
- A `reminder_time` inside quiet hours returns `422 validation_failed` with the field named.
- At most 2 notifications a day and none in quiet hours are enforced by the scheduler, not by this route.

POST and DELETE /api/push/subscribe

```
POST /api/push/subscribe
{ "endpoint": "https://fcm.googleapis.com/fcm/send/...", "keys": { "p256dh":
"...", "auth": "..." } }
```

Returns `201` on a new subscription and `200` when the endpoint is already stored. `DELETE` with `{ "endpoint": "..." }` returns `204`. When a push service answers 410 the server removes the subscription itself and falls back to email (FR-NOT-03).

GET /api/export

Returns everything the user owns, including mind data (FR-SET-02).

- `?format=json` returns one JSON document with every entity above, journal bodies decrypted for the owner.
- `?format=csv` returns a ZIP with one CSV per entity.
- The file is streamed and never stored. Limited to 3 requests a day per user (proposed).

DELETE /api/account

```
DELETE /api/account
{ "confirm": "DELETE" }
```

```
202 Accepted
{ "delete_requested_at": "2026-10-10T16:50:00Z", "purge_after": "2026-10-
17T16:50:00Z" }
```

Access ends at once: the session is revoked and later calls return `401`. The data is purged by a nightly job, targeted within 7 days and never later than 30 days (FR-SET-03, NFR-07). A wrong or missing `confirm` returns `422`. Offering to cancel a deletion is not in the requirements and is left out.

6. Scheduled jobs and billing

/api/cron/*

Internal routes called by the scheduler, not by the app. Each requires `Authorization: Bearer <CRON_SECRET>` and returns `401` otherwise. Every job is safe to run twice, because each one checks what it has already done.

Route	Frequency	What it does	Requirements
/api/cron/reminders	Every 5 minutes	Finds users whose reminder time falls in the window in their time zone; sends push, or email as fallback; respects 2 a day and quiet hours; sends one re-engagement after 3 days without a log, then none for 7 days	FR-NOT-01 to 04, 06
/api/cron/close-days	Hourly	Evaluates ended days for limit goals, flags goals past their deadline as `due` (extend, shrink or restart), and applies the low-mood rule	FR-GOL-08, FR-MON-11, FR-MND-08
/api/cron/recurring	Daily	Posts recurring expenses due that day; on the last day of a short month it posts items set for day 29 to 31	FR-MON-09
/api/cron/purge	Nightly	Deletes accounts past `purge_after`, expired tombstones (90 days), applied ops (30 days), old notification logs and used magic links	FR-SET-03, NFR-07
/api/cron/recap	Sunday evening, V1	Builds the weekly recap and sends it	FR-ENG-10

Reminder text is chosen from the tone library and never contains journal text or exact money amounts (FR-NOT-06). The scheduler service is an open decision; hourly or finer schedules may need an external scheduler depending on the hosting plan.

/api/billing/* (V1, Plus plan)

Not part of the MVP. The price and plan limits are still to be decided after beta feedback, so only the shape is fixed here.

Route	Purpose
`POST /api/billing/orders`	Create a Razorpay order for the Plus plan; accepts an `Idempotency-Key` header; returns the order id the app passes to the Razorpay checkout
`POST /api/billing/webhook`	Razorpay calls this; the server verifies the `X-Razorpay-Signature` HMAC (SHA-256) against the raw body, then sets `users.plan`; unsigned or invalid calls return `400`
`GET /api/billing/status`	Current plan and renewal state for the Me screen

Webhook events are stored by Razorpay event id so a repeated event changes nothing. TIKO never holds, moves or links to the user's money, so these routes concern only the subscription fee.

7. Errors and rate limits

Every error uses one shape. Messages are plain and never blame the user; the app decides what to show.

```
{ "error": { "code": "validation_failed", "message": "Target must be greater than 0.",
             "details": { "field": "target" }, "retryable": false } }
```

HTTP	`code`	When	Retry
400	`bad_request`	Malformed JSON or missing header	No
401	`unauthenticated`	No or expired session	After sign-in
403	`forbidden`	Origin check failed or the row belongs to another user	No
404	`not_found`	Unknown route or id	No

HTTP	`code`	When	Retry
410	`cursor_too_old`	Pull cursor older than tombstone retention	Full resync
413	`payload_too_large`	More than 200 ops or over 1 MB	Split the batch
422	`validation_failed`	A field fails its rule	No, fix the data
426	`client_too_old`	`schema_v` below the supported minimum	After reload
429	`rate_limited`	A limit below was exceeded; `Retry-After` header in seconds	After the wait
500	`server_error`	Unexpected failure	Yes, with backoff
503	`unavailable`	Maintenance or provider outage	Yes, with backoff

Sync routes return `200` even when single mutations are rejected; use the per-mutation `status` and `error.code` from section 3. A failed or interrupted sync never deletes data on the device, so the client retries with backoff capped at 30 seconds.

Rate limits

Scope	Limit	Source
Magic link requests	5 per 10 minutes per email, 20 per hour per IP	Technical Specification
Write routes (push, merge-guest, PATCH, subscribe)	120 per minute per user	Technical Specification
Pull	300 per minute per user	Proposed
Export	3 per day per user	Proposed
Account deletion	5 per hour per user	Proposed

Limits are counted in a Redis-style store, not the main database. Normal use stays far under them: a person logging at full speed makes one push every few seconds, and the client batches the outbox.

Security notes. All traffic is HTTPS with HSTS. Request bodies and journal fields are scrubbed from Sentry events. Analytics events carry no journal text, notes or exact amounts. Cron and webhook routes authenticate by secret or signature, never by cookie.